Introduction

The **gem5** simulator is a modular platform for computer-system architecture research, encompassing system-level architecture as well as processor microarchitecture. It is widely recognized in academia and industry for its ability to simulate a variety of instruction set architectures (ISAs) including X86, ARM, RISC-V, MIPS, and POWER (**The gem5 About, n.d.)**.

Key features include (The gem5 Project, n.d.):

- Gem5 offers interchangeable CPU models ranging from simple one CPI to detailed out of order designs and KVM-based virtualization.
- Support event driven memory system capable of modeling complex hierarchies and diverse technologies like HBM and DDR.
- Facilitates research on homogeneous and heterogeneous multi core topologies using the MOESI snooping cache coherence protocol.
- Allows instantiation of multiple systems within a single simulation to model complex client server network interactions.

Environment Setup

Gem5 is architected primarily for Linux systems. To guarantee stability and performance, the simulator is frequently validated against Ubuntu 22.04 and Ubuntu 24.04 (**The gem5 Project, n.d.**).

Also we need below dependencies to install gem5:

- **git** : gem5 uses git for version control.
- **gcc**: gcc is used to compiled gem5. **Version >=10 must be used**. We support up to gcc Version 13.
- **SCons** : gem5 uses SCons as its build environment. SCons 3.0 or greater must be used.
- **Python 3.6+** : gem5 relies on Python development libraries. gem5 can be compiled and run in environments using Python 3.6+.

Installation Instruction gem5 on Windows 10

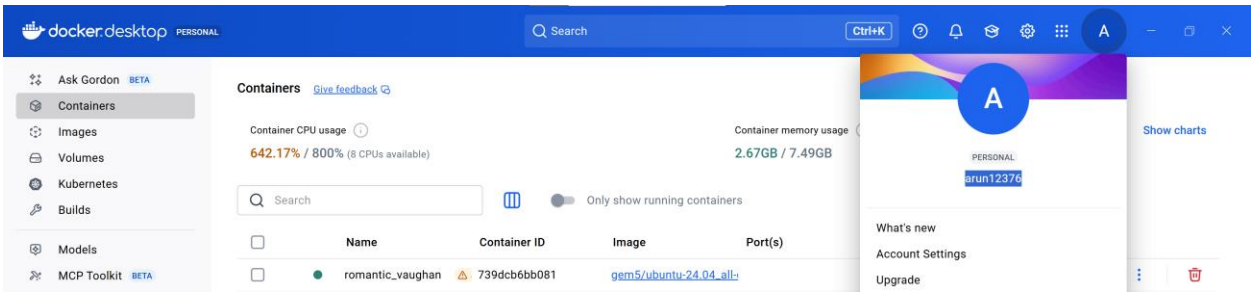
Since gem5 is built for Linux, the best way to run it on Windows is by using **WSL2** to create a Linux-like environment. The official guide actually recommends using a **Docker image** for the installation. I've already got Docker Desktop set up on my laptop and it's ready to go, so I can start pulling the image now (https://www.gem5.org/documentation/general_docs/building)

Note → This document does not include all steps required to install WSL2 and docker desktop, follow below link to install WSL2 and docker desktop if you have window 10 and above.

https://learn.microsoft.com/en-us/windows/wsl/install

https://docs.docker.com/desktop/setup/install/windows-install/

Once docker is up and running it will show like below screenshot, I used same user which is tied to my github.



Clone the gem5 repo, make sure you have git client to clone this with command

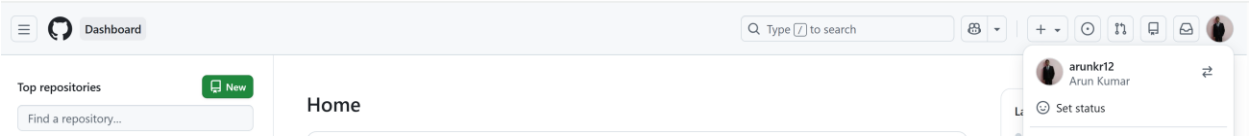
GitHub Account creation

Creating a GitHub account is a straightforward process:
Go to GitHub.com and click the **Sign-up** button. The site uses a "terminal-style" interface where it will ask you for three things in order:

- **Email address:** Use one you check often, as you'll need to verify it.
- **Password:** Make it strong (at least 15 characters, or 8 characters with a number and lowercase letter).

- **Username:** Pick something professional; this will be part of your profile URL (e.g., github.com/yourname).

  Your account will look like below



Now you may clone the gem5 git repo in your local director. Use below command

- ➢ git clone https://github.com/gem5/gem5

```
Windows PowerShell
PS D:\Gem5> git clone https://github.com/gem5/gem5
Cloning into 'gem5'...
remote: Enumerating objects: 299973, done.
remote: Counting objects: 100% (149/149), done.
remote: Compressing objects: 100% (88/88), done.
Receiving objects: 100% (299973/299973), 173.53 MiB |

Resolving deltas: 100% (228840/228840), done.
Updating files: 100% (10933/10933), done.
PS D:\Gem5>
```

Before moving forward, open **PowerShell** and run these two quick checks:

- **Check your CPU cores:** Type $env:NUMBER_OF_PROCESSORS to see how many processors you have.
- **Verify Python:** Run python -V to make sure Python is installed and ready to use.

```
PS D:\Gem5> cd .\gem5\
PS D:\Gem5\gem5> $env:NUMBER_OF_PROCESSORS
8
PS D:\Gem5\gem5> python -V
Python 3.7.4
PS D:\Gem5\gem5> $
```

To grab the gem5 image, use the docker pull command. For example, if you want the **Ubuntu 24.04** version that comes pre-loaded with all the optional dependencies, just run:

- **docker pull ghcr.io/gem5/ubuntu-24.04_all-dependencies:v24-0**

```
PS D:\Gem5\gem5> docker pull  ghcr.io/gem5/ubuntu-24.04_all-dependencies:v24-0
v24-0: Pulling from gem5/ubuntu-24.04_all-dependencies
00d679a470c4: Pull complete
c782a11a41b6: Pull complete
Digest: sha256:90aeaca1f108ac1169ae37cff2fd0c03c7f757d7a4f15e568ebff5b5a812828d
Status: Downloaded newer image for ghcr.io/gem5/ubuntu-24.04_all-dependencies:v24-0
ghcr.io/gem5/ubuntu-24.04_all-dependencies:v24-0

What's next:
    View a summary of image vulnerabilities and recommendations → docker scout quick
PS D:\Gem5\gem5>
```

Now we will launc a pre-configured Linux environment that links your local folder (D:\Gem5\gem5) directly into the docker container so you can build and run gem5.

- **docker run --volume D:\Gem5\gem5:/gem5 --workdir /gem5 --rm -it ghcr.io/gem5/ubuntu-24.04_all-dependencies:v24-0**

To Run as root user

```
PS D:\Gem5\gem5> docker run --volume D:\Gem5:/gem5 --workdir /gem5 --rm -it ghcr.io/gem5/ubuntu-24.04_all-dependencies:v24-0
WARNING: The requested image's platform (linux/arm64) does not match the detected host platform (linux/amd64/v3) and no specific
root@739dcb6bb081:/gem5#
```

Once your docker container is up and running, we could run scons build with X86 with opt. Run below command from gem5 directory.

- **scons build/X86/gem5.opt -j 8**

```
root@739dcb6bb081:/gem5# ls
bin  lib  share
root@739dcb6bb081:/gem5# cd gem5/
root@739dcb6bb081:/gem5/gem5# ls
CODE-OF-CONDUCT.md  COPYING     LICENSE           README.md         SConstruct  build        build_tools  docs  include                    pyproject.toml    site_scons  system  util
CONTRIBUTING.md     KCONFIG.md  MAINTAINERS.yaml  RELEASE-NOTES.md  TESTING.md  build_opts   configs      ext   optional-requirements.txt  requirements.txt  src         tests
root@739dcb6bb081:/gem5/gem5# scons build/X86/gem5.opt -j 8
scons: Reading SConscript files ...
Mkdir("/gem5/gem5/build/X86/gem5.build")
Checking for linker -Wl,--as-needed support... yes
Checking for compiler -gz support... yes
Checking for linker -gz support... yes
Info: Using Python config: python3-config
Checking for C header file Python.h... yes
Checking Python version... 3.12.3
Checking for accept(0,0,0) in C++ library None... yes
Checking for zlibVersion() in C++ library z... yes
Checking for C library tcmalloc_minimal... yes
Building in /gem5/gem5/build/X86
"build_tools/kconfig_base.py" "/gem5/gem5/build/X86/gem5.build/Kconfig" "/gem5/gem5/src/Kconfig"
Checking size of struct kvm_xsave ... yes
Checking for C header file fenv.h... yes
Checking for C header file png.h... yes
Checking for clock_nanosleep(0,0,NULL,NULL) in C library None... yes
Checking for C header file valgrind/valgrind.h... no
Checking for pkg-config package hdf5-serial... yes
Checking for H5Fcreate("", 0, 0, 0) in C library hdf5... yes
Checking for H5::H5File("", 0) in C++ library hdf5_cpp... yes
Checking for C header file capstone/capstone.h... yes
Checking for C header file linux/kvm.h... yes
Checking for timer_create(CLOCK_MONOTONIC, NULL, NULL) in C library None... yes
Checking for member exclude_host in struct perf_event_attr...yes
Checking for C header file linux/if_tun.h... yes
Checking for shm_open("/test", 0, 0) in C library None... yes
Checking for pkg-config package protobuf... yes
Checking for backtrace_symbols_fd((void *)1, 0, 0) in C library None... yes
"ext/Kconfiglib/defconfig.py" --kconfig "/gem5/gem5/build/X86/gem5.build/Kconfig" "/gem5/gem5/build_opts/X86"
Loaded configuration '/gem5/gem5/build_opts/X86'
Configuration saved to '/gem5/gem5/build/X86/gem5.build/config'
```

It will keep running for next couple of hours and logging like below

```
[     CXX] X86/mem/Bridge.py.cc [Ob] .pyo
[EMBED PY] src/python/m5/ext/pystats/statistic.py [Ob] X86/python/m5/ext/pystats/statistic.py.cc
[EMBED PY] src/arch/x86/X86NativeTrace.py [Ob] X86/arch/x86/X86NativeTrace.py.cc
[EMBED PY] src/python/gem5/components/cachehierarchies/chi/nodes/abstract_node.py [Ob] X86/python/gem5/components/cachehierarchies/chi/nodes/abstract_node.py.cc
[     CXX] X86/python/m5/ext/pystats/statistic.py.cc [Ob] .pyo
[     CXX] X86/arch/x86/X86NativeTrace.py.cc [Ob] .pyo
[     CXX] X86/python/gem5/components/cachehierarchies/chi/nodes/abstract_node.py.cc [Ob] .pyo
[     CXX] X86/arch/x86/X86NativeTrace.py.cc [Ob] .o
[EMBED PY] src/python/gem5/resources/client_api/client_query.py [Ob] X86/python/gem5/resources/client_api/client_query.py.cc
[EMBED PY] src/python/m5/simulate.py [Ob] X86/python/m5/simulate.py.cc
[EMBED PY] src/dev/serial/Uart.py [Ob] X86/dev/serial/Uart.py.cc
[     CXX] X86/python/gem5/resources/client_api/client_query.py.cc [Ob] .pyo
[     CXX] X86/python/m5/simulate.py.cc [Ob] .pyo
[     CXX] X86/dev/serial/Uart.py.cc [Ob] .pyo
[EMBED PY] src/python/gem5/components/cachehierarchies/ruby/caches/viper/dma_controller.py [Ob] X86/python/gem5/components/cachehierarchies/ruby/caches/viper/dma_controller.py.cc
[     CXX] X86/python/gem5/components/cachehierarchies/ruby/caches/viper/dma_controller.py.cc [Ob] .pyo
[EMBED PY] src/cpu/CPUTracers.py [Ob] X86/cpu/CPUTracers.py.cc
[ TRACING] [Ob] X86/debug/Event.hh
[     CXX] X86/cpu/CPUTracers.py.cc [Ob] .pyo
[EMBED PY] src/python/m5/core.py [Ob] X86/python/m5/core.py.cc
[EMBED PY] src/python/gem5/components/boards/experimental/lupv_board.py [Ob] X86/python/gem5/components/boards/experimental/lupv_board.py.cc
[     CXX] X86/python/m5/core.py.cc [Ob] .pyo
[     CXX] X86/python/gem5/components/boards/experimental/lupv_board.py.cc [Ob] .pyo
[EMBED PY] src/python/gem5/components/processors/simple_core.py [Ob] X86/python/gem5/components/processors/simple_core.py.cc
[     CXX] X86/python/gem5/components/processors/simple_core.py.cc [Ob] .pyo
[EMBED PY] src/mem/HMCController.py [Ob] X86/mem/HMCController.py.cc
[EMBED PY] src/mem/ruby/structures/RubyCache.py [Ob] X86/mem/ruby/structures/RubyCache.py.cc
[EMBED PY] src/python/gem5/components/devices/gpus/amdgpu.py [Ob] X86/python/gem5/components/devices/gpus/amdgpu.py.cc
[     CXX] X86/mem/HMCController.py.cc [Ob] .pyo
[     CXX] X86/mem/ruby/structures/RubyCache.py.cc [Ob] .pyo
[EMBED PY] src/python/m5/params/network_params.py [Ob] X86/python/m5/params/network_params.py.cc
[     CXX] X86/python/gem5/components/devices/gpus/amdgpu.py.cc [Ob] .pyo
[     CXX] X86/python/m5/params/network_params.py.cc [Ob] .pyo
[EMBED PY] src/sim/power/MathExprPowerModel.py [Ob] X86/sim/power/MathExprPowerModel.py.cc
[     CXX] X86/sim/power/MathExprPowerModel.py.cc [Ob] .pyo
[EMBED PY] src/python/gem5/utils/multiprocessing/_command_line.py [Ob] X86/python/gem5/utils/multiprocessing/_command_line.py.cc
[EMBED PY] src/python/m5/util/multidict.py [Ob] X86/python/m5/util/multidict.py.cc
[     CXX] X86/python/gem5/utils/multiprocessing/_command_line.py.cc [Ob] .pyo
[     CXX] X86/python/m5/util/multidict.py.cc [Ob] .pyo
```

Once finished it will log like below screen shot. If it display “scons: done building targets” means you scons build is successful and ready for simulation.

```
[    SHCC] ext/softfloat/s_normRoundPackToF128.c -> X86/ext/softfloat/s_normRoundPackToF128.os
[    SHCC] ext/softfloat/s_normSubnormalExtF80Sig.c -> X86/ext/softfloat/s_normSubnormalExtF80Sig.os
[    SHCC] ext/softfloat/s_normSubnormalF128Sig.c -> X86/ext/softfloat/s_normSubnormalF128Sig.os
[    SHCC] ext/softfloat/s_propagateNaNF128UI.c -> X86/ext/softfloat/s_propagateNaNF128UI.os
[    SHCC] ext/softfloat/s_roundPackToExtF80.c -> X86/ext/softfloat/s_roundPackToExtF80.os
[    SHCC] ext/softfloat/s_roundPackToF128.c -> X86/ext/softfloat/s_roundPackToF128.os
[    SHCC] ext/softfloat/s_roundToUI64.c -> X86/ext/softfloat/s_roundToUI64.os
[    SHCC] ext/softfloat/s_shiftRightJam128.c -> X86/ext/softfloat/s_shiftRightJam128.os
[    SHCC] ext/softfloat/s_shiftRightJam128Extra.c -> X86/ext/softfloat/s_shiftRightJam128Extra.os
[    SHCC] ext/softfloat/s_shiftRightJam256M.c -> X86/ext/softfloat/s_shiftRightJam256M.os
[    SHCC] ext/softfloat/s_shiftRightJam64Extra.c -> X86/ext/softfloat/s_shiftRightJam64Extra.os
[    SHCC] ext/softfloat/s_shortShiftLeft128.c -> X86/ext/softfloat/s_shortShiftLeft128.os
[    SHCC] ext/softfloat/s_shortShiftRight128.c -> X86/ext/softfloat/s_shortShiftRight128.os
[    SHCC] ext/softfloat/s_shortShiftRightJam64Extra.c -> X86/ext/softfloat/s_shortShiftRightJam64Extra.os
[    SHCC] ext/softfloat/s_shortShiftRightJam128.c -> X86/ext/softfloat/s_shortShiftRightJam128.os
[    SHCC] ext/softfloat/s_shortShiftRightJam128Extra.c -> X86/ext/softfloat/s_shortShiftRightJam128Extra.os
[    SHCC] ext/softfloat/s_subMagsExtF80.c -> X86/ext/softfloat/s_subMagsExtF80.os
[    SHCC] ext/softfloat/s_subMagsF128.c -> X86/ext/softfloat/s_subMagsF128.os
[    SHCC] ext/softfloat/s_sub128.c -> X86/ext/softfloat/s_sub128.os
[    SHCC] ext/softfloat/s_sub256M.c -> X86/ext/softfloat/s_sub256M.os
[    SHCC] ext/softfloat/ui32_to_extF80.c -> X86/ext/softfloat/ui32_to_extF80.os
[    SHCC] ext/softfloat/ui32_to_f128.c -> X86/ext/softfloat/ui32_to_f128.os
[    SHCC] ext/softfloat/ui64_to_extF80.c -> X86/ext/softfloat/ui64_to_extF80.os
[    SHCC] ext/softfloat/ui64_to_f128.c -> X86/ext/softfloat/ui64_to_f128.os
[      AR] -> X86/ext/softfloat/libsoftfloat.a
[  RANLIB] -> X86/ext/softfloat/libsoftfloat.a
scons: done building targets.
root@739dcb6bb081:/gem5/gem5#
```

Verify build is success and can do the simulation

- **/build/X86/gem5.opt –version**

Troubleshooting

When running first time scons build command it failed with permission error

```
PS D:\Gem5\gem5> docker run --volume D:\Gem5\gem5:/gem5 --workdir /gem5 --rm -it ghcr.io/gem5/ubuntu-20.04_all-dependencies:v23-0
root@4c014fefa6dc:/gem5# ls
CODE-OF-CONDUCT.md  COPYING     LICENSE           README.md         SConstruct  build       build_tools  docs  include                    pyproject.toml    site_scons  system  util
CONTRIBUTING.md     KCONFIG.md  MAINTAINERS.yaml  RELEASE-NOTES.md  TESTING.md  build_opts  configs      ext   optional-requirements.txt  requirements.txt  src         tests
root@4c014fefa6dc:/gem5# scons build/X86/gem5.opt -j 8
scons: Reading SConscript files ...

You're missing the pre-commit/commit-msg hooks. These hook help to ensure your
code follows gem5's style rules on git commit and your commit messages follow
our commit message requirements. This script will now install these hooks in
your .git/hooks/ directory.
Press enter to continue, or ctrl-c to abort:
 /bin/sh: 1: util/pre-commit-install.sh: not found
It is strongly recommended you install the pre-commit hooks before working with gem5. Do you want to continue compilation (y/n)?
y
Mkdir("/gem5/build/X86/gem5.build")
Checking for linker -Wl,--as-needed support... yes
Warning: Detected GCC version 9.4.0 is not officially supported.
         gem5 supports GCC v11 up to v14.2.
Checking for compiler -gz support... yes
Checking for linker -gz support... yes
Info: Using Python config: python3-config
Checking for C header file Python.h... yes
Checking Python version... 3.8.10
Checking for accept(0,0,0) in C++ library None... yes
Checking for zlibVersion() in C++ library z... yes
Checking for C library tcmalloc_minimal... yes
Building in /gem5/build/X86
"build_tools/kconfig_base.py" "/gem5/build/X86/gem5.build/Kconfig" "/gem5/src/Kconfig"
/usr/bin/env: ‘python3\r’: No such file or directory
scons: *** Error 127
Error: Failed to build base Kconfig file
root@4c014fefa6dc:/gem5# find util build_tools -type f -exec sed -i 's/\r$//' {} +
root@4c014fefa6dc:/gem5# chmod +x build_tools/kconfig_base.py
root@4c014fefa6dc:/gem5# chmod +x util/pre-commit-install.sh
root@4c014fefa6dc:/gem5#
```

Run below command and re run your scons again

- **Chmod +x build_tools/kconfig_base.py**
- **Chmod +x util/pre-commit-install.sh**

Even after finish I can't find the gem5.opt file in build/X86 directory. Re run

- **scons build/X86/gem5.opt --ignore-errors**

```
root@739dcb6bb081:/gem5/gem5# scons build/X86/gem5.opt --ignore-errors
scons: Reading SConscript files ...
Mkdir("/gem5/gem5/build/X86/gem5.build")
Checking for linker -Wl,--as-needed support... (cached) yes
Checking for compiler -gz support... (cached) yes
Checking for linker -gz support... (cached) yes
Info: Using Python config: python3-config
Checking for C header file Python.h... (cached) yes
Checking Python version... (cached) 3.12.3
Checking for accept(0,0,0) in C++ library None... (cached) yes
Checking for zlibVersion() in C++ library z... (cached) yes
Checking for C library tcmalloc_minimal... (cached) yes
Building in /gem5/gem5/build/X86
"build_tools/kconfig_base.py" "/gem5/gem5/build/X86/gem5.build/Kconfig" "/gem5/gem5/src/Kconfig"
Checking size of struct kvm_xsave ... (cached) yes
Checking for C header file fenv.h... (cached) yes
Checking for C header file png.h... (cached) yes
Checking for clock_nanosleep(0,0,NULL,NULL) in C library None... (cached) yes
Checking for C header file valgrind/valgrind.h... (cached) no
Checking for pkg-config package hdf5-serial... (cached) yes
Checking for H5Fcreate("", 0, 0, 0) in C library hdf5... (cached) yes
Checking for H5::H5File("", 0) in C++ library hdf5_cpp... (cached) yes
Checking for C header file capstone/capstone.h... (cached) yes
Checking for C header file linux/kvm.h... (cached) yes
Checking for timer_create(CLOCK_MONOTONIC, NULL, NULL) in C library None... (cached) yes
Checking for member exclude_host in struct perf_event_attr...(cached) yes
Checking for C header file linux/if_tun.h... (cached) yes
Checking for shm_open("/test", 0, 0) in C library None... (cached) yes
Checking for pkg-config package protobuf... (cached) yes
Checking for backtrace_symbols_fd((void *)1, 0, 0) in C library None... (cached) yes
Checking whether __i386__ is declared... (cached) no
Checking whether __x86_64__ is declared... (cached) yes
BUILD_TLM not set, not building CHI-TLM integration

Checking for compiler -Wno-self-assign-overloaded support... (cached) yes
Checking for linker -Wno-free-nonheap-object support... (cached) no
scons: done reading SConscript files.
scons: Building targets ...
 [VER TAGS]     -> X86/sim/tags.cc
scons: done building targets.
root@739dcb6bb081:/gem5/gem5#
```

I was still getting same issue so I tried the Force Link

This tells SCons to forget what it thinks it finished

1. Clear the database just to be safe

- rm -f .sconsign.dblite

2. Force the build of the specific binary

- scons build/X86/gem5.opt -j 1

After that it was successful.

Reference List

Gem5 Getting Started

The gem5 Project. (n.d.). *Getting started*. Retrieved from https://www.gem5.org/getting_started/

Building gem5

The gem5 Project. (n.d.). *Building gem5*. Retrieved from https://www.gem5.org/documentation/general_docs/building

Docker Desktop Installation (Windows)

Docker Documentation. (n.d.). *Install Docker Desktop on Windows*. Retrieved from https://docs.docker.com/desktop/setup/install/windows-install/

WSL2 Installation (Windows)

https://learn.microsoft.com/en-us/windows/wsl/install